

Part I Documentation: GPT-Style Decoder-Only Transformer

Causal Self-Attention and Next-Token Prediction

Anatolie Jentimir

May 6, 2026

Abstract

This report documents the implementation and testing of a small GPT-style decoder-only Transformer model for character-level next-token prediction. The project focuses on the first part of the assignment: building an autoregressive language model using causal self-attention, training it on a text dataset, saving model artifacts, and generating text from a prompt. A quick CPU training script was added to make the project reproducible and easy to test without requiring long training time or a GPU.

1 Project Goal

The goal of this project is to implement and test a decoder-only Transformer similar in structure to a small GPT model. The model receives a sequence of previous tokens and predicts the next token. Because the model is autoregressive, each token is allowed to attend only to itself and previous tokens, not to future tokens.

The main objectives are:

- implement causal self-attention;
- train the model using the next-token prediction objective;
- evaluate the model using validation loss and perplexity;
- save model artifacts for reproducibility;
- test text generation from a starting prompt.

2 Model Overview

The implemented model is a small decoder-only Transformer. It uses token embeddings, positional embeddings, masked self-attention blocks, feed-forward layers, normalization, and a final linear projection to predict the next character/token.

2.1 Decoder-Only Architecture

A decoder-only Transformer processes an input sequence:

$$x_1, x_2, \dots, x_T$$

and predicts the next-token targets:

$$x_2, x_3, \dots, x_{T+1}.$$

The model learns the probability distribution:

$$P(x_{t+1} \mid x_1, x_2, \dots, x_t).$$

This is the standard autoregressive language modeling setup. The model does not use an encoder because the task is not sequence-to-sequence translation. Instead, it learns to continue text based only on previous context.

3 Causal Self-Attention

The most important part of the decoder-only model is causal self-attention. In normal self-attention, every token could attend to every other token. For language generation, that would be wrong because the model would be able to look at future answers during training.

To prevent information leakage, a causal mask is applied. The mask blocks attention from position t to any future position $j > t$. The attention computation is:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V$$

where M is the causal mask. Future positions receive a very negative value before the softmax, forcing their attention probability close to zero.

This means token t can only use:

$$x_1, x_2, \dots, x_t$$

and cannot use:

$$x_{t+1}, x_{t+2}, \dots, x_T.$$

4 Training Objective

The model is trained using the next-token prediction objective. For each input sequence, the target is the same sequence shifted one position to the left.

For example, if the input sequence is:

ROMEO:

then the model learns to predict each next character after seeing the previous characters.

The loss function is cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log P(y_i \mid x_{\leq i})$$

where y_i is the correct next token and $P(y_i | x_{\leq i})$ is the probability assigned by the model.

5 Quick CPU Training Script

The original full epoch training script was too slow and did not show progress frequently enough on CPU. To solve this, a quick training script was added. Instead of training by full epochs, it trains for a fixed number of random mini-batch steps. This makes the project easier to test on a normal laptop.

The quick training script provides:

- fixed-step training instead of full-epoch training;
- visible progress heartbeat messages;
- validation checks during training;
- automatic best-model checkpoint saving;
- final result files for the report;
- generation test after training.

The training batch file runs the quick training script and then tests generation. This makes the project easier for the instructor or another user to reproduce.

6 How to Run the Project

From the project directory, activate the Python environment if needed and run:

```
training.bat
```

The batch file starts quick CPU training and then runs a generation test.

A direct Python command can also be used:

```
python train_quick_cpu.py --steps 1200
```

After training, generated files are stored in the `outputs` directory.

7 Saved Artifacts

The quick training run saves the following artifacts:

These files make the project reproducible because they record the trained model, configuration, tokenizer, and results.

8 Training Results

The model was tested with a quick CPU run for 1200 steps. During training, the validation negative log-likelihood and perplexity improved significantly.

Figure 1 shows the training and validation loss curves from the quick CPU run. Both curves drop quickly during the early training steps and then stabilize near the final range, which supports that the model is learning from the dataset rather than remaining random.

The final log reported:

Table 1: Saved training artifacts

File	Purpose
outputs/best_model.pt	Best saved model checkpoint
outputs/config.json	Model and training configuration
outputs/tokenizer.json	Character vocabulary/token mapping
outputs/training_log.csv	Training and validation metrics
outputs/final_results.json	Final summary of best metrics
outputs/loss_plot.png	Plot of loss/perplexity trends

Table 2: Quick CPU training result summary

Metric	Initial / Earlier Value	Best Final Value
Validation NLL	approximately 4.3241	2.4855
Validation Perplexity	approximately 75.50	12.01

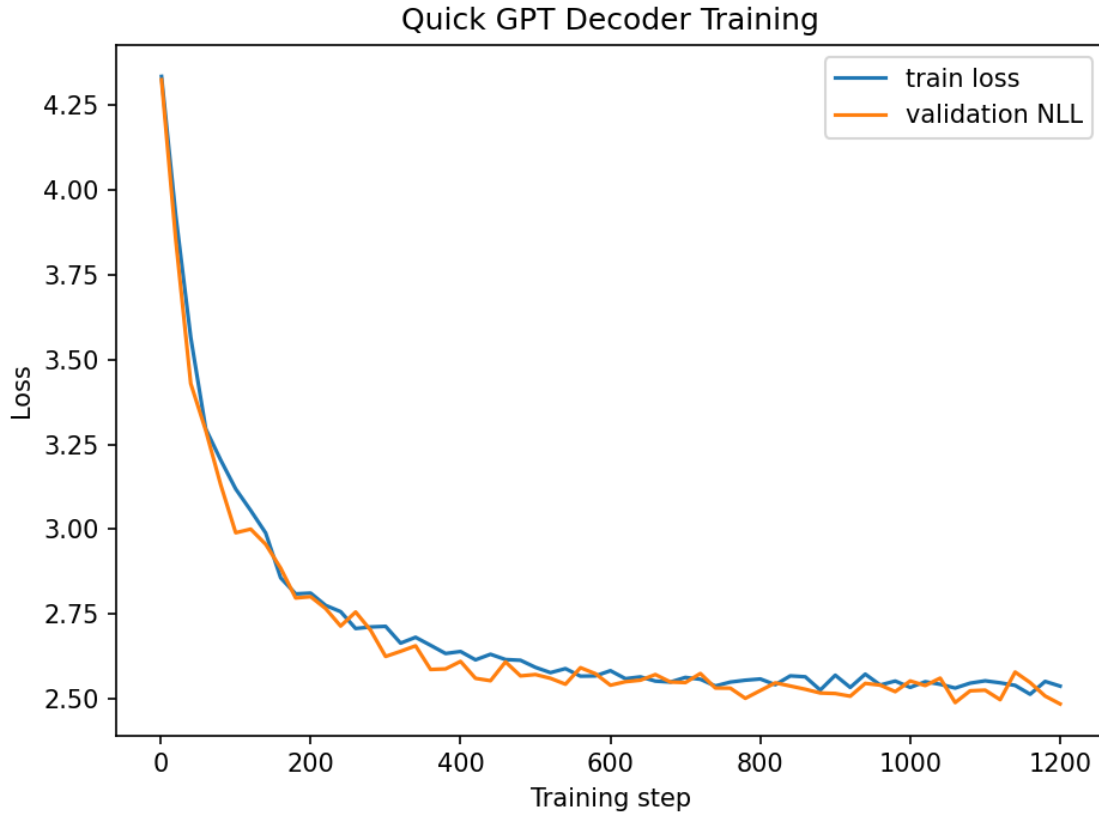


Figure 1: Quick GPT decoder training loss curve for the 1200-step CPU run.

```
Best validation NLL: 2.4855
Best validation PPL: 12.01
Saved artifacts to: outputs
```

This confirms that the model was learning. A lower negative log-likelihood means the model assigned higher probability to correct next tokens. A lower perplexity means the model became less uncertain about the next character prediction.

9 Generation Test

After training, the model was tested using a prompt beginning with `ROMEO:`. The generated output was not perfect English, but it showed that the complete pipeline works: the trained checkpoint can be loaded, the prompt can be encoded, the model can autoregressively sample next tokens, and the generated tokens can be decoded back into text.

Example generated text from the quick run:

```
ROMEO:
TRI:
MGO:
D ce cendoulrthiat be, bne hrino ou ir dofourthite ive berel,

Whye t to s wimas lllf ck reare re itonouleas lir ha f inat toreara sthar y
```

The output is expected to be noisy because the model is intentionally small and trained briefly on CPU. The purpose of the quick run is reproducibility and verification, not production-quality text generation.

10 Discussion

The training results show that the decoder-only Transformer implementation is functioning correctly. The validation perplexity decreased from about 75.50 to 12.01, which is a strong sign that the model learned useful character-level patterns from the dataset.

The quick CPU training version is useful for a homework submission because it avoids long full-epoch training and gives immediate evidence that the code works. It also prints regular progress messages so the user can see that training is active.

For better generated text, the model could be trained longer, increased in size, or trained on a larger dataset. However, for this assignment, the current result is enough to demonstrate the core concepts:

- causal masking prevents future-token leakage;
- next-token prediction trains the language model;
- the validation loss and perplexity improve during training;
- the model can save checkpoints and generate text.

11 Limitations

The current implementation is intentionally small and CPU-friendly. Because of that, the generated text is still noisy and not fully coherent. The main limitations are:

- small number of trainable parameters;

- character-level tokenization instead of subword tokenization;
- short training time;
- CPU-only training;
- limited dataset size.

These limitations are acceptable for a classroom demonstration because the goal is to show a working decoder-only Transformer training pipeline, not to build a large production language model.

12 Conclusion

This project successfully implements and tests a GPT-style decoder-only Transformer for next-token prediction. The model uses causal self-attention to prevent access to future tokens and learns through cross-entropy loss. A quick CPU training script was added to make the project reproducible and easy to test. The final quick run achieved a best validation NLL of 2.4855 and a best validation perplexity of 12.01, confirming that the training process works correctly.

A Training Evidence Screenshot

Figure 2 shows the terminal output from the successful training run. The screenshot confirms that the model completed 1200 training steps, saved the best checkpoint, created the output artifacts, and ran a generation test.

```
GPT Decoder Quick CPU Train × + ▾
heartbeat step 1170/1200 | last loss = 2.6301 | lr = 0.000001 | elapsed = 26.4s
heartbeat step 1175/1200 | last loss = 2.5356 | lr = 0.000001 | elapsed = 26.5s
heartbeat step 1180/1200 | last loss = 2.4983 | lr = 0.000001 | elapsed = 26.6s
EVAL step 1180/1200 | train loss = 2.5519 | val NLL = 2.5088 | val PPL = 12.29
heartbeat step 1185/1200 | last loss = 2.4701 | lr = 0.000000 | elapsed = 26.7s
heartbeat step 1190/1200 | last loss = 2.4504 | lr = 0.000000 | elapsed = 26.8s
heartbeat step 1195/1200 | last loss = 2.5210 | lr = 0.000000 | elapsed = 26.9s
heartbeat step 1200/1200 | last loss = 2.5162 | lr = 0.000000 | elapsed = 27.0s
EVAL step 1200/1200 | train loss = 2.5380 | val NLL = 2.4855 | val PPL = 12.01
Saved new best model: outputs/best_model.pt

Done.
Best validation NLL: 2.4855
Best validation PPL: 12.01
Saved artifacts to: outputs

Training finished. Files created:
outputs\best_model.pt
outputs\config.json
outputs\tokenizer.json
outputs\training_log.csv
outputs\final_results.json
outputs\loss_plot.png

Testing generation now...
ROMEO:
TRI:
MGO:
D ce cendoulrthiat be, bne hrino ou ir dofourthite ive berel,

Whye t to s wimas lllf ck reare re itonouleas lir ha f inat toreara sthar y

GUThe t chath fore I t terongoume wiavyo tous, ll

Done.
Press any key to continue . . .
```

Figure 2: Terminal output from the successful quick CPU training run.